ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Membre de HONORIS UNITED UNIVERSITIES

3ème année Ingénierie Informatique et Réseaux (IIR)

Module : Programmation Python et Framework

MINI-PROJET PYTHON

Simulation d'une Plateforme E-Learning Simple

(NEXUS LEARN)

Réalisé par :

Rim CHARAI

Nizar EL IDRYSY

Israe ZAHAAR

Encadré par :

Mme. Soundouss ABROUN

Année Universitaire 2025 – 2026

Table des matières

Introduction **2**

1 Description du Système et des Acteurs **3**

1.1 Description du système choisi 3

1.2 Description des acteurs 3

2 Structure des Données et Fonctionnalités **4**

2.1 La structure des données utilisée 4

2.2 La description des fonctions principales 5

2.2.1 Module d'Authentification (`authentification.py`) 5

2.2.2 Opérations CRUD - Module Responsable (`responsable.py`) 5

2.2.3 Supervision globale - Module Admin (`admin.py`) 5

3 Captures d'Écran et Interfaces **6**

3.1 Captures d'écran illustrant le fonctionnement 6

Conclusion **8**

Introduction

Dans le cadre de notre formation en développement, il est crucial de maîtriser les fondements de la programmation back-end avant d'aborder des frameworks complexes tels que Django. Ce mini-projet a pour objectif de développer une simulation d'application web en ligne de commande (CLI) en utilisant le langage Python.

Le présent rapport documente la conception et le développement de **NEXUS LEARN**, une plateforme minimaliste de gestion d'apprentissage (E-Learning). Ce document suit rigoureusement le cahier des charges en présentant le système choisi, la description des acteurs, la structure des données, les fonctionnalités principales implémentées, ainsi que des captures d'écran illustrant son fonctionnement.

Chapitre 1

Description du Système et des Acteurs

1.1 Description du système choisi

Conformément au projet du cahier des charges, nous avons opté pour le développement d'une **Mini Plateforme E-Learning Simple**.

Dans un contexte éducatif de plus en plus numérisé, cette application permet de simuler un environnement où les enseignants peuvent structurer et diffuser des cours, tandis que les étudiants peuvent consulter le catalogue de formations. L'application est développée de manière modulaire (répartie sur plusieurs fichiers `.py`) et utilise une interface en ligne de commande enrichie de couleurs ANSI pour une meilleure expérience utilisateur.

1.2 Description des acteurs

Le système met en interaction trois types d'acteurs, chacun disposant de permissions et de menus spécifiques :

- **L'Administrateur :** Il possède un accès global au système. Son rôle principal est la gouvernance des données. Il peut consulter l'intégralité de la base de données (liste de tous les étudiants et professeurs) et supprimer des comptes en cas de besoin.
- **L'Enseignant (Responsable) :** Il est le créateur de contenu. Il dispose d'un espace personnel lui permettant d'ajouter de nouveaux cours, de modifier les informations de ses propres cours, et de les supprimer. Le système garantit qu'un enseignant ne peut interagir qu'avec les ressources qui lui appartiennent.
- **L'Étudiant (Utilisateur) :** Il est le consommateur du contenu. Une fois authentifié, il peut parcourir le catalogue complet des cours disponibles sur la plateforme afin de consulter les titres et les descriptions des enseignements.

Chapitre 2

Structure des Données et Fonctionnalités

2.1 La structure des données utilisée

Pour simuler une base de données sans utiliser de SGBDR (comme SQLite ou MySQL), nous avons exploité le module `json` de Python. Les données sont stockées dans le fichier `nexus_db.json` et chargées en mémoire dans le module `data.py`.

Conformément aux directives, la structure repose sur des **listes contenant des dictionnaires imbriqués**. L'identifiant (ID) unique de chaque entité est utilisé comme clé principale.

Listing 2.1 – Exemple de la structure des données utilisée

```python
# Structure chargee en memoire dans data.py
administrateurs = [
    {1: {"nom": "Admin Principal", "email": "admin@emsi.ma", "password": "admin"}}
]

enseignants = [
    {1: {"nom": "reem", "email": "reem", "password": "1234"}}
]

etudiants = [
    {1: {"nom": "rim charai", "email": "rim", "password": "123"}}
]

cours = [
    {1: {"titre": "Algorithmique", "description": "Bases", "enseignant_id": 1}}
]
```

La présence de la clé `"enseignant_id"` dans la liste des cours simule une clé étrangère

(Foreign Key), permettant de lier chaque cours à son créateur.

2.2 La description des fonctions principales

Le code a été structuré en modules distincts pour séparer la logique fonctionnelle :

2.2.1 Module d'Authentification (`authentification.py`)

— `connexion()` : Gère le processus de login. Elle limite l'utilisateur à **3 tentatives** maximum. En parcourant les listes, elle identifie le type d'acteur et renvoie son rôle et son ID.

— `inscription()` : Permet de créer un compte étudiant ou professeur.

— `generer_id_specifique(liste)` : Un algorithme crucial qui analyse les clés des dictionnaires existants pour attribuer automatiquement un ID unique (ID max + 1) au nouvel utilisateur ou au nouveau cours.

2.2.2 Opérations CRUD - Module Responsable (`responsable.py`)

Ce module gère le contenu pédagogique.

— `ajouter_cours(id_prof)` : Ajoute un dictionnaire dans la liste `data.cours` en y attachant l'ID du professeur connecté.

— `modifier_cours(id_prof)` et `supprimer_cours(id_prof)` : Parcourent la liste des cours, vérifient que le cours appartient bien au professeur (`info["enseignant_id"] == id_prof`), et appliquent la modification ou la suppression correspondante.

2.2.3 Supervision globale - Module Admin (`admin.py`)

— `supprimer_utilisateur()` : Permet à l'administrateur de cibler une liste (étudiants ou professeurs) et de supprimer un dictionnaire utilisateur via son ID, mettant à jour le fichier JSON via la fonction `data.sauvegarder()`.

Chapitre 3

Captures d'Écran et Interfaces

Afin d'améliorer l'expérience utilisateur (UX) dans le terminal, nous avons développé un module `ui.py` gérant le nettoyage de l'écran (`clear_screen`) et l'affichage des couleurs dynamiques (ANSI).

3.1 Captures d'écran illustrant le fonctionnement

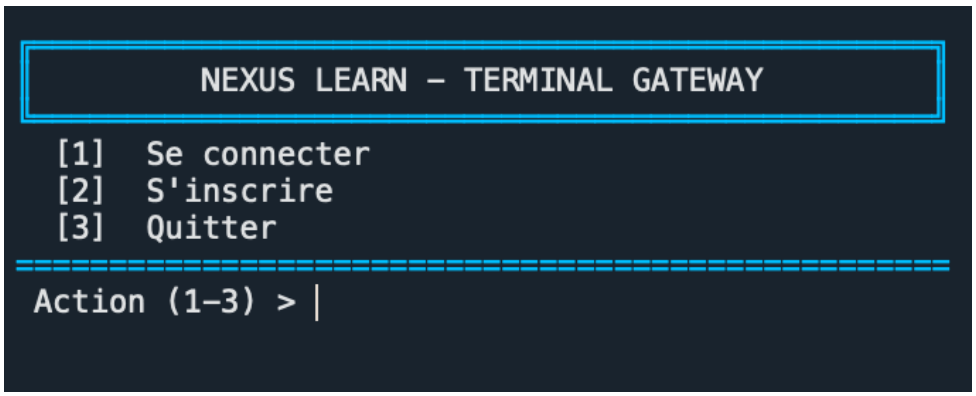

FIGURE 3.1 – Écran de démarrage avec la bannière ASCII personnalisée

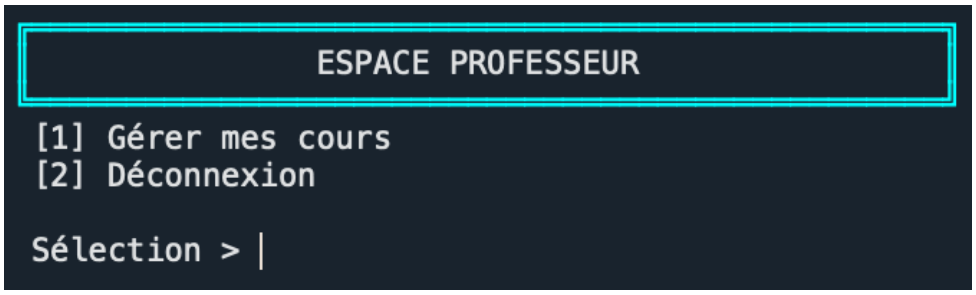

FIGURE 3.2 – Menu principal de l'Espace Professeur

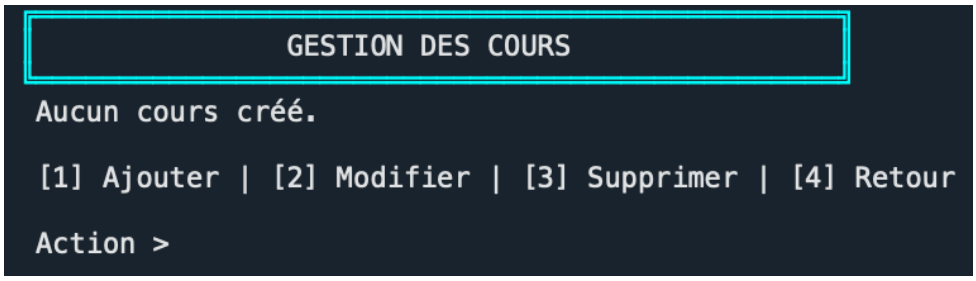

FIGURE 3.3 – Interface de gestion des cours (Opérations CRUD)

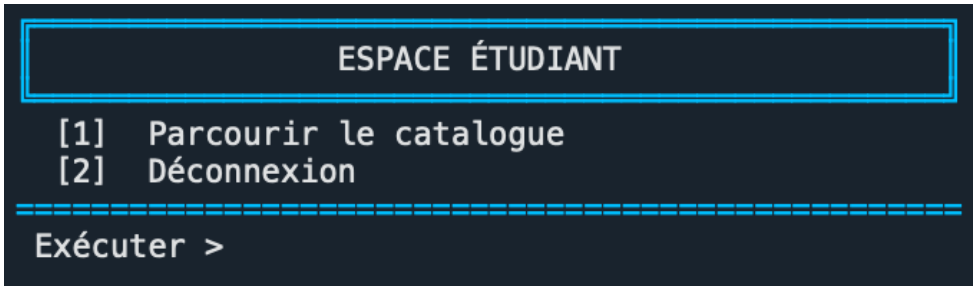

FIGURE 3.4 – Espace Étudiant

Conclusion

La réalisation de cette plateforme E-Learning en ligne de commande nous a permis de consolider nos acquis en algorithmique et en programmation modulaire sous Python.

La contrainte d'utiliser des listes de dictionnaires imbriqués pour simuler une base de données nous a poussés à réfléchir rigoureusement aux relations entre les entités (clés primaires et clés étrangères). L'implémentation de la sécurité (limitation des tentatives, restriction par rôle) et des opérations CRUD répondent aux exigences d'une architecture MVT (Model-View-Template) simplifiée.

Ce projet constitue une excellente préparation technique et méthodologique avant d'entamer le développement d'applications web complexes avec le framework Django.